

Questão 1 [2 pontos]:

São dadas duas listas **ordenadas** de tamanho m e n . Dê um algoritmo de tempo $O(\log m + \log n)$ para computar o k -ésimo menor elemento da união das duas listas.

Questão 2 [2 pontos]:

Suponha que você tenha k arrays ordenados $A_1, A_2, \dots, A_k$, cada um com n elementos. Você deseja combiná-los em um único array ordenado de tamanho kn . Uma maneira de fazer isso seria usar a operação de intercalação (*merge*). Sabendo que a intercalação de dois array X e Y precisa de $|X| + |Y|$ passos.

1. Calcule quantos passos o algoritmo a seguir leva: Primeiro intercale os arrays A_1, A_2 , depois intercale o resultado com A_3 e assim por diante.
2. Projete um algoritmo melhor para este problema.

Questão 3 [2 pontos]:

Dado um conjunto S de strings, encontre de forma eficiente o prefixo comum mais longo. Exemplos:

Entrada: $S = \{\text{"instituto", "instigar", "instável", "instaurar"}\}$, Saída: "inst"

Entrada: $S = \{\text{"portuguese", "português", "portugal"}\}$, Saída: "portug"

Questão 4 [2 ponto]:

Considere o seguinte problema.

Entrada: Grafo não direcionado $G = (V, E)$; pesos de arestas w_e ; subconjunto de vértices $U \subset V$.

Saída: A Árvore Geradora mais leve na qual os nós de U são folhas (pode haver outras folhas na árvore também).

Questão 5 [2 ponto]:

Considere os dois algoritmos gulosos para o CAIXEIRO VIAJANTE. Mostre que ambos não são exatos, isto é, não garantem retornar a solução ótima.

Algoritmo 1 (Algoritmo do vizinho mais próximo)

1. Inicialize todos os vértices como não visitados.
2. Selecione um vértice arbitrário, defina-o como o vértice atual u . Marque u como visitado.
3. Descubra a aresta mais curta conectando o vértice atual u e um vértice não visitado v .
4. Defina v como o vértice atual u . Marque v como visitado.
5. Se todos os vértices do domínio forem visitados, então termine. Caso contrário, vá para o passo 3.

Algoritmo 2 (Algoritmo da inserção mais barata)

1. Crie um ciclo C com três vértices quaisquer.
2. Escolha um vértice k qualquer fora de C .
3. Encontre uma aresta $(i, j) \in C$ tal que $d_{ik} + d_{kj} - d_{ij}$ é mínimo.
4. Adicione k a C , remova a aresta (i, j) e adicione as arestas (i, k) e (k, j) .
5. Se C contiver todos os nós, pare. Caso contrário, vá para 2.

Questão 6 [2 pontos]:

Um ladrão entra em uma loja e vê um conjunto I de n itens, $I = \{a_1, a_2, \dots, a_n\}$. Cada item tem um peso associado w_i e um valor v_i . Idealmente, o ladrão gostaria de roubar tudo para obter o máximo benefício. No entanto, há muito que ele pode carregar. O ladrão tem uma mochila com capacidade K . O ladrão agora tem que determinar quais itens roubar para que seu peso total não exceda K e seu valor total seja máximo.

1. Suponha que os itens sejam tais que uma fração de um item possa ser retirada, ou seja, o ladrão pode pegar alguma parte de um item (por exemplo, $0,4 w_i$ de a_i) e deixar a parte restante. Isso é chamado de Problema da Mochila Fracionada. Seja $A \subseteq I$ o subconjunto de itens que o ladrão rouba. Elabore um algoritmo guloso $O(n \log n)$ para encontrar o subconjunto A tal que $\sum_{a_i \in A} w_i < K$ e $\sum_{a_i \in A} v_i$ é máximo.
2. Suponha que os itens só possam ser retirados como um todo, ou seja, o ladrão só pode pegar ou deixar um item; ele não pode pegar uma fração de um item. Isso é chamado de Problema da Mochila Binária (ou Mochila 0-1). Seu algoritmo guloso para a versão fracionária do problema (pergunta anterior) ainda encontra uma solução ótima? Justifique sua resposta.